🚀 Collections Framework Masterclass

Chapter 10 — TreeSet (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`TreeSet` is the Set implementation to know when the requirement is:

> **Unique elements + sorted order + range/navigation operations**

Unlike `HashSet` and `LinkedHashSet`, `TreeSet` does **not** use hashing.

Its core is a **balanced Red-Black Tree**.

---

1. What is TreeSet?

`TreeSet` is a sorted implementation of the `Set` interface.

Package:

```
java.util.TreeSet
```

Declaration:

```
public class TreeSet<E>
    extends AbstractSet<E>
    implements NavigableSet<E>,
               Cloneable,
               Serializable
```

Hierarchy:

```
Iterable
   ↓
Collection
   ↓
Set
   ↓
SortedSet
```

```
   ↓
NavigableSet
   ↓
TreeSet
```

The important interfaces are:

```
SortedSet
NavigableSet
```

They provide functionality beyond a normal Set.

---

2. Key Characteristics

Property	TreeSet
Duplicates	❌
Ordering	Sorted
Index-based access	❌
Typical operations	O(log n)
Internal structure	Red-Black Tree
Null	Generally not supported
Thread-safe	❌
Supports range operations	✅
Supports navigation	✅

---

3. The Main Difference from HashSet

Recall:

```
HashSet
→ uniqueness + fast lookup
```

```
LinkedHashSet
→ uniqueness + insertion order
```

```
TreeSet
→ uniqueness + sorted order
```

Example:

```
Set<Integer> set = new TreeSet<>();

set.add(50);
set.add(10);
set.add(40);
set.add(20);
```

Iteration:

```
10
20
40
50
```

The elements are automatically ordered.

---

4. How Does TreeSet Maintain Order?

This is the key internal concept.

TreeSet uses a tree-based structure backed by `TreeMap`.

Conceptually:

```
TreeSet
   ↓
TreeMap
   ↓
Red-Black Tree
```

The TreeSet stores its elements as keys in the underlying sorted map structure.

5. Why a Red-Black Tree?

A normal Binary Search Tree can become unbalanced.

Example:

```
10

   20

      30

         40
```

This becomes effectively a linked list.

Search becomes:

```
O(n)
```

A Red-Black Tree automatically maintains approximate balance.

Conceptually:

```
       20
      /
    10    30

             40
```

Height remains O(log n).

Therefore:

```
search   → O(log n)
insert   → O(log n)
delete   → O(log n)
```

---

6. What is a Red-Black Tree?

A Red-Black Tree is a **self-balancing Binary Search Tree** with additional color information.

Each node is conceptually:

```
+----------------+
| Key            |
| Left           |
| Right          |
| Parent         |
| Color          |
+----------------+
```

Color:

```
RED
or
BLACK
```

The coloring rules ensure the tree doesn't become excessively unbalanced.

You don't need to memorize the complete balancing algorithm unless specifically asked, but you should understand why it exists.

---

7. TreeSet Ordering

There are two ways to define ordering:

1. Natural ordering

Using:

```
Comparable
```

Example:

```
TreeSet<Integer> set =
    new TreeSet<>();
```

Integers already implement `Comparable`.

---

2. Custom ordering

Using:

```
Comparator
```

Example:

```
TreeSet<Integer> set =
    new TreeSet<>(
        Comparator.reverseOrder()
    );
```

Now:

```
50
40
30
20
10
```

---

8. Comparable vs Comparator

This is a major interview topic.

Comparable

The class defines its own natural ordering.

```
class Employee
    implements Comparable<Employee> {

    @Override
```

```
    public int compareTo(Employee other) {
        return Integer.compare(
            this.id,
            other.id
        );
    }
}
```

Usage:

```
TreeSet<Employee> employees =
    new TreeSet<>();
```

Comparator

Ordering is supplied externally.

```
Comparator<Employee> byName =
    Comparator.comparing(
        Employee::getName
    );

TreeSet<Employee> employees =
    new TreeSet<>(byName);
```

---

9. TreeSet Doesn't Use equals() to Determine Uniqueness

This is one of the **most important TreeSet interview questions**.

For ordering and duplicate determination, TreeSet uses:

```
compareTo()
```

or:

```
Comparator.compare()
```

If:

```
compare(a, b) == 0
```

TreeSet treats them as equivalent for Set purposes.

Therefore the element is not added as a separate entry.

---

10. TreeSet Duplicate Example

```
TreeSet<Integer> set =
    new TreeSet<>();

set.add(10);
set.add(10);
```

The second `10` isn't added because:

```
compare(10,10) == 0
```

---

11. The Big Difference: HashSet vs TreeSet

HashSet:

```
hashCode()
+
equals()
```

TreeSet:

```
compareTo()
```

or:

```
Comparator.compare()
```

This distinction is extremely important.

---

12. Dangerous Comparator Example

Suppose:

```
class Employee {

    int id;
    String name;
}
```

Comparator:

```
Comparator<Employee> comparator =
    Comparator.comparing(
        Employee::getName
    );
```

Now:

```
Employee 101 → John
Employee 102 → John
```

Comparator returns:

```
0
```

TreeSet considers them equivalent.

Only one may be stored.

Even though their IDs differ.

---

13. Important Rule

Your ordering should normally be **consistent with equality**.

Ideally:

```
compare(a,b) == 0
```

should correspond to:

```
a.equals(b) == true
```

If they are inconsistent, TreeSet can still operate according to the comparator, but its Set behavior may surprise you.

---

14. Natural Ordering Example

```
TreeSet<String> set =
    new TreeSet<>();

set.add("Zebra");
set.add("Apple");
set.add("Mango");
set.add("Banana");
```

Result:

```
Apple
Banana
Mango
Zebra
```

Strings use their natural ordering.

---

15. NavigableSet

TreeSet implements `NavigableSet`.

This gives us powerful navigation methods.

Important methods:

```
lower()
floor()
ceiling()
higher()
```

These are frequently asked in interviews.

---

16. lower()

Returns the greatest element **strictly less than** the given element.

```
TreeSet<Integer> set =
    new TreeSet<>(
        Arrays.asList(10,20,30,40)
    );

set.lower(30);
```

Result:

```
20
```

---

17. floor()

Returns the greatest element **less than or equal to** the given element.

```
set.floor(30);
```

Result:

```
30
```

For:

```
set.floor(35);
```

Result:

```
30
```

---

18. higher()

Returns the smallest element **strictly greater than** the given element.

```
set.higher(30);
```

Result:

```
40
```

---

19. ceiling()

Returns the smallest element **greater than or equal to** the given element.

```
set.ceiling(30);
```

Result:

```
30
```

For:

```
set.ceiling(35);
```

Result:

```
40
```

---

20. Easy Way to Remember

For:

```
10 20 30 40
```

Around `30`:

```
lower(30)   → 20
floor(30)   → 30

ceiling(30) → 30
higher(30)  → 40
```

Think:

```
lower   < target
floor   <= target

ceiling >= target
higher  > target
```

---

21. first() and last()

Returns smallest and largest elements.

```
set.first();
set.last();
```

For:

```
10,20,30,40
```

Result:

```
first() → 10
last()  → 40
```

Complexity:

```
O(log n)
```

---

22. pollFirst() and pollLast()

Unlike `first()` and `last()`, these **remove** the element.

```
set.pollFirst();
```

Removes the smallest element.

```
set.pollLast();
```

Removes the largest element.

---

23. Range Views

One of TreeSet's biggest advantages over HashSet.

Important methods:

```
subSet()
headSet()
tailSet()
```

---

24. subSet()

Returns a range view.

```
TreeSet<Integer> set =
    new TreeSet<>(
        Arrays.asList(
            10,20,30,40,50
        )
    );

set.subSet(20, 50);
```

Result:

```
20,30,40
```

By default:

```
from → inclusive
to   → exclusive
```

---

25. Inclusive/Exclusive Version

You can explicitly control boundaries:

```
set.subSet(
    20, true,
    50, true
);
```

Result:

```
20,30,40,50
```

---

26. headSet()

Returns elements before a specified point.

```
set.headSet(30);
```

Result:

```
10,20
```

By default, `30` is excluded.

Can explicitly include it:

```
set.headSet(30, true);
```

Result:

```
10,20,30
```

---

27. tailSet()

Returns elements from a specified point onward.

```
set.tailSet(30);
```

Result:

```
30,40,50
```

By default, `30` is included.

---

28. Important: Range Methods Return Views

This is an excellent interview question.

Consider:

```
NavigableSet<Integer> subset =
    set.subSet(20, 50);
```

The subset is generally a **view backed by the original TreeSet**, not an independent copy.

Therefore changes to the view can affect the original Set, subject to the range restrictions.

---

29. Null Handling

Generally, TreeSet does not allow `null` when using natural ordering.

Example:

```
TreeSet<Integer> set =
    new TreeSet<>();

set.add(null);
```

This results in:

```
NullPointerException
```

Why?

The tree needs to compare elements to determine their position.

There is no natural ordering between `null` and an `Integer`.

---

30. Can TreeSet Store Null with a Comparator?

Potentially, yes, if the supplied comparator explicitly defines how `null` should be ordered.

Example:

```
TreeSet<String> set =
    new TreeSet<>(
        Comparator.nullsFirst(
            Comparator.naturalOrder()
```

```
        )
    );
```

Now `null` can be ordered before non-null values.

Interview nuance:

> "TreeSet does not allow null" is a good default statement for natural ordering, but a suitable Comparator can define null ordering.

---

31. Custom Objects

Suppose:

```
class Employee {

    int id;
    String name;
}
```

This will not work automatically:

```
TreeSet<Employee> set =
    new TreeSet<>();

set.add(new Employee());
```

unless Employee provides natural ordering or the TreeSet has a Comparator.

Otherwise Java cannot determine:

```
Employee A
    vs
Employee B
```

and a comparison-related exception can occur.

---

32. Using Comparable

```
class Employee
        implements Comparable<Employee> {

    int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(
            this.id,
            other.id
        );
    }
}
```

Now:

```
TreeSet<Employee> employees =
    new TreeSet<>();

employees.add(new Employee(103));
employees.add(new Employee(101));
employees.add(new Employee(102));
```

Iteration:

```
101
102
103
```

---

33. Using Comparator

You don't need to modify Employee.

```
TreeSet<Employee> employees =
    new TreeSet<>(
        Comparator.comparingInt(
            e -> e.id
        )
    );
```

This is often preferable when you need different sorting strategies.

For example:

```
By ID
By Name
By Salary
By Joining Date
```

Each can have a different Comparator.

---

34. TreeSet and Mutable Objects

Another advanced interview trap.

Suppose the ordering depends on:

```
employee.salary
```

You insert the Employee.

Later:

```
employee.salary = 900000;
```

The object's ordering key changed.

But the tree structure wasn't automatically rebuilt.

Now operations may behave unexpectedly.

Therefore:

Don't mutate fields that determine ordering while the object is stored in a TreeSet.

---

35. Thread Safety

TreeSet is not thread-safe.

For synchronized access:

```
SortedSet<Integer> set =
    Collections.synchronizedSortedSet(
        new TreeSet<>()
    );
```

For concurrent sorted collections, Java provides specialized options such as:

```
ConcurrentSkipListSet
```

We'll discuss concurrent collections later.

---

36. Performance

Typical complexity:

Operation	Complexity
add()	O(log n)
remove()	O(log n)
contains()	O(log n)
first()	O(log n)
last()	O(log n)
lower()	O(log n)
floor()	O(log n)
ceiling()	O(log n)
higher()	O(log n)
iteration	O(n)

TreeSet sacrifices HashSet's average O(1) lookup in exchange for **sorted order and navigation**.

---

37. HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	❌	❌	❌
Ordering	None guaranteed	Insertion	Sorted
Internal structure	Hash table	Hash table + links	Red-Black Tree
Average/Typical add	O(1)	O(1)	O(log n)
contains	O(1) avg	O(1) avg	O(log n)
Range operations	❌	❌	✅
Navigation	❌	❌	✅
Null	One	One	Generally no
Memory	Lower	Higher	Tree-node overhead

---

38. When Should You Use TreeSet?

Use TreeSet when you need:

```
Unique elements
      +
Sorted order
      +
Navigation / range queries
```

Examples:

Sorted scores

```
TreeSet<Integer> scores;
```

Sorted timestamps

```
TreeSet<Instant> timestamps;
```

Range queries

```
All values between 100 and 500
```

Nearest-value lookup

```
Find the smallest value >= X
```

This is exactly what `ceiling()` provides.

---

39. When NOT to Use TreeSet?

Only need uniqueness + fast lookup

Use:

```
HashSet
```

Need insertion order

Use:

```
LinkedHashSet
```

Need duplicates

Use:

```
List
```

Need queue/stack behavior

Use:

```
ArrayDeque
```

---

40. ⭐ Interview Scenario

Requirement

You maintain employee salaries.

You need:

- No duplicates
- Sorted salaries
- Find the next salary greater than ₹500,000

Best choice?

```
TreeSet<Integer> salaries;
```

Then:

```
salaries.higher(500000);
```

---

41. ⭐ Interview Scenario

Requirement

You have millions of IDs.

Need:

- Unique IDs
- Fast membership check
- No ordering

Use:

```
HashSet<Long>
```

Not TreeSet.

---

42. ⭐ Interview Scenario

Requirement

You receive transaction IDs and need:

- Remove duplicates
- Preserve first-seen order

Use:

```
LinkedHashSet<Long>
```

Not TreeSet.

---

43. Common Interview Traps

Trap 1

> TreeSet uses `hashCode()` to detect duplicates.

❌ Wrong.

It uses:

```
compareTo()
```

or:

```
Comparator.compare()
```

Trap 2

> TreeSet always allows null.

❌ Wrong.

Natural ordering generally does not support null.

A Comparator can explicitly define null ordering.

Trap 3

> TreeSet preserves insertion order.

❌ Wrong.

It preserves **sorted order**.

Trap 4

> TreeSet is faster than HashSet.

❌ Generally wrong.

HashSet has O(1) average basic operations.

TreeSet provides O(log n) operations but offers sorting and navigation.

Trap 5

> `compareTo() == 0` always means `equals() == true`.

❌ Not necessarily.

But for sorted collections, if the comparator returns zero, TreeSet treats the elements as equivalent for Set purposes.

---

44. Most Important Concept

Memorize this:

```
HashSet
   ↓
```

```
hashCode() + equals()
   ↓
Uniqueness
```

```
TreeSet
   ↓
compareTo() / Comparator
   ↓
Ordering + uniqueness
```

That distinction explains most TreeSet interview questions.

---

45. Frequently Asked Interview Questions

Q1. How does TreeSet work internally?

It uses a balanced tree-based structure, specifically a Red-Black Tree through `TreeMap` infrastructure.

Q2. Why is TreeSet O(log n)?

Because the Red-Black Tree maintains logarithmic height.

Q3. How does TreeSet determine duplicates?

If comparison returns zero:

```
compareTo() == 0
```

or:

```
Comparator.compare() == 0
```

TreeSet treats the elements as equivalent for Set purposes.

Q4. Does TreeSet use equals()?

`equals()` is not the primary mechanism for ordering/duplicate determination. The comparator or natural ordering determines equivalence.

Q5. What happens if objects don't implement Comparable?

If no Comparator is supplied, TreeSet cannot establish natural ordering and insertion can fail with a comparison-related exception.

Q6. What is the difference between `lower()` and `floor()`?

```
lower(x) → greatest element < x

floor(x) → greatest element <= x
```

Q7. Difference between `higher()` and `ceiling()`?

```
higher(x)  → smallest element > x

ceiling(x) → smallest element >= x
```

Q8. What is the difference between first() and pollFirst()?

```
first()
→ returns smallest element
→ does not remove

pollFirst()
→ returns smallest element
→ removes it
```

Q9. What is the difference between TreeSet and HashSet?

HashSet optimizes for fast average lookup without ordering.

TreeSet provides sorted ordering and navigation at O(log n).

Q10. Can TreeSet store custom objects?

Yes, if:

- They implement `Comparable`, or
- A compatible `Comparator` is provided.

---

46. ⭐⭐⭐⭐⭐ Quick Coding Question

What is the output?

```
TreeSet<Integer> set =
    new TreeSet<>();

set.add(10);
set.add(20);
set.add(30);
set.add(40);

System.out.println(set.lower(25));
System.out.println(set.floor(20));
System.out.println(set.ceiling(25));
System.out.println(set.higher(30));
```

Answer:

```
20
20
30
40
```

---

47. ⭐⭐⭐⭐⭐ Tricky Coding Question

What happens?

```
TreeSet<String> set =
    new TreeSet<>(
        Comparator.comparingInt(String::length)
    );
```

```
set.add("Java");
set.add("Code");
set.add("Spring");
```

`"Java"` and `"Code"` have the same length.

The comparator returns:

```
0
```

for them.

Therefore TreeSet considers them equivalent for Set purposes.

Result contains only one of those equal-length strings, plus `"Spring"`.

This is a classic interview trap.

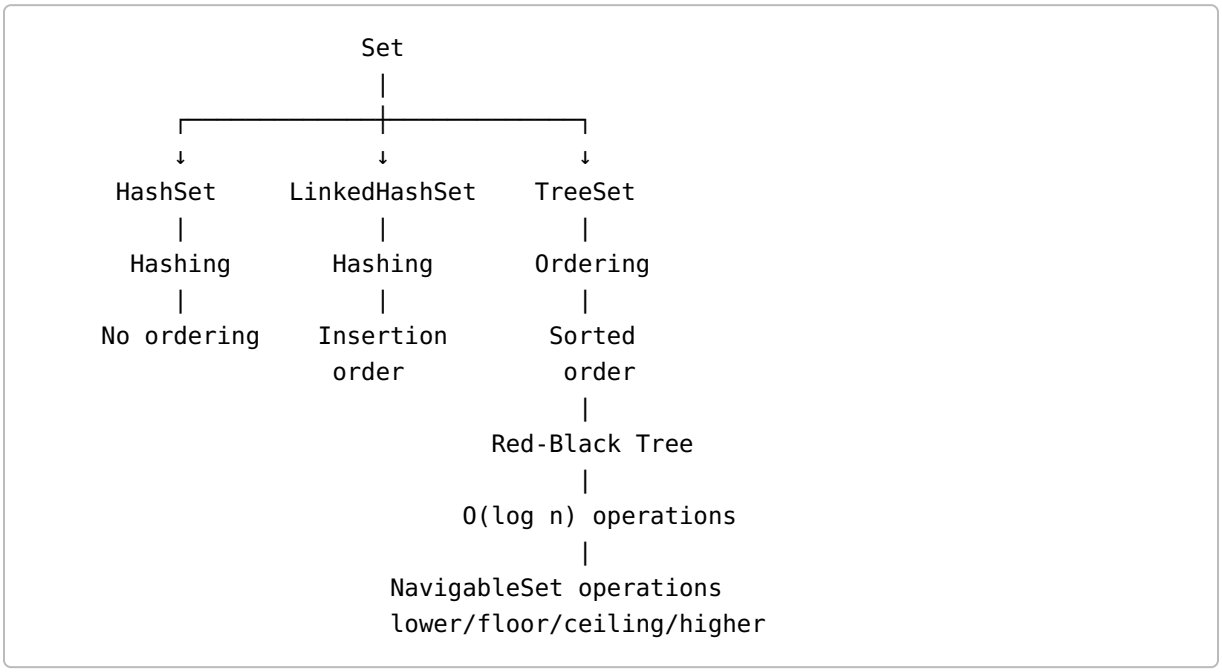
---

48. Decision Framework

When choosing a Set:

```
Need uniqueness?
       ↓
      YES
       ↓
Need sorted order?
   ↙           ↘
 YES           NO
 ↓              ↓
TreeSet      Need insertion
             order?
             ↙       ↘
           YES       NO
            ↓         ↓
     LinkedHashSet  HashSet
```

🧠 Final Mental Model



🔥 Chapter Summary

Concept	TreeSet
Interface	`NavigableSet`
Duplicates	Not allowed
Ordering	Sorted
Internal structure	Red-Black Tree
Backing infrastructure	TreeMap
`add()`	O(log n)
`contains()`	O(log n)
`remove()`	O(log n)
Navigation	Yes
Range queries	Yes
Natural ordering	`Comparable`
Custom ordering	`Comparator`

Concept	TreeSet
Null	Generally unsupported with natural ordering
Thread-safe	No

🎯 Interview Readiness Checklist

You should be able to answer:

- [ ] Why does TreeSet use a Red-Black Tree?
- [ ] How does TreeSet determine duplicates?
- [ ] TreeSet vs HashSet?
- [ ] TreeSet vs LinkedHashSet?
- [ ] Comparable vs Comparator?
- [ ] What happens when `compareTo()` returns 0?
- [ ] Why can TreeSet reject null?
- [ ] What are `lower()`, `floor()`, `ceiling()`, and `higher()`?
- [ ] Difference between `first()` and `pollFirst()`?
- [ ] What are `subSet()`, `headSet()`, and `tailSet()`?
- [ ] Why are TreeSet operations O(log n)?
- [ ] What happens if a TreeSet's ordering field is mutated?
- [ ] Why can two objects that are not `equals()` be treated as duplicates by TreeSet?

🚀 Next Chapter

Chapter 11 — Queue Interface (Complete Deep Dive)

We'll focus on what's new:

- Queue vs Stack
- FIFO
- Queue method pairs
- `add()` vs `offer()`
- `remove()` vs `poll()`
- `element()` vs `peek()`
- Blocking vs non-blocking queues
- PriorityQueue
- ArrayDeque
- LinkedList as Queue

- Choosing the correct Queue implementation
- Real-world producer/consumer scenarios
- 20+ interview questions